Expected Linear Time Algorithm for Computing 3D Relative Neighborhood Graphs

Z. Zzyzek[a,*,1] (Researcher)

ARTICLE INFO

Keywords:
graph algorithm
relative neighborhood graph
computation geometry
RNG

ABSTRACT

An expected linear time algorithm for computing the relative neighborhood graph (RNG) for points in Euclidean space, distributed uniformly in a unit three dimensional cube, is given. For each point, potential neighbors for an RNG connection are limited to an expected small bounding volume. Volumes relative to the considered point are bounded through half plane partitions, constructed from the point and its neighbors, that remove regions where no RNG edge connections to the point can occur. The small bounding volume provides an expected constant number of neighbors to consider, making each single point RNG calculation $O(1)$, providing the $O(n)$ expected runtime when enumerating through all points. To the author's knowledge, this improves on the previously best known $O(n \log n)$ in three dimensions.

1. Introduction

Let $P = (p_0, p_1, \ldots, p_{n-1})$ be a set of points in $\mathbb{R}^3$. The relative neighborhood graph, RNG(P), is an undirected, simple graph that connects two vertices, p_i, p_j if and only if there is no other vertex within the *lune* between them.

$$(p_i, p_j) \in \mathrm{RNG}(P) \iff |p_i - p_j| \leq \max_{k \neq i,j}(|p_i - p_k|, |p_j - p_k|)$$

Figure **??** shows a highlighted 2D example of a connections and precluded connection and Figure **??**, **??** shows examples of a relative neighborhood graph for 2D and 3D.

When points are placed uniformly at random in the unit cube, the maximum degree of the relative neighborhood graph is bounded by a constant. When random in this manner, the edges from any point connecting it to its relative neighborhood graph only extend to a proportionally small local neighborhood. This local neighborhood is effectively constant in neighbor size, allowing us to calculate each points relative neighborhood graph edges in constant time, providing a linear time algorithm to find the relative neighborhood graph in 3D.

2. Related Work

OUTLINE

- Talk about KN85 and KNT86 and their exceptions to corner and side regions of grid. Also talk about restriction to 2d and how generalizations to 3d are complicated

- 3d algs of agarwal,matousek are meant for worst case setups but super linear (e.g. advesarial)

- talk about efficient computation of Urquhart (sp?) graph

- talk about using delaunay triangulation to bootstrap

- review gives best known algorithm at $O(n \log n)$ for general position (doesn't talk about random position?)

- Jaromczyk and Kowaluk on $O(n^2 \log n)$ 3d algorithm with 'coarse' elimination phase that was inspiration for plane cut heuristic

3. Motivation and Structure

3.1. Overview

Let $P = (p_0, p_1, \ldots, p_{n-1})$, where each $p_i \in \mathbb{R}^3$ is chosen uniformly at random in the unit cube.

A rough overview of the algorithm proceeds as follows:

- **Initialize** a three dimensional grid, where each cell size of $n^{-1/3}$ on a side
- **Bin** points into grid cells, allowing for multiple points in a bin.
- For each point, p:
  - **Add** nearby points from adjacent grid cells
  - **Increase** the radius of a bounding cube, at grid cell increments, until the collected nearby points have precluded connections from all other points outside bounding cube to p
  - **Extend** the list of nearby points, once a bounding cube has been found, to include points that might preclude connections on the inner bounding cube
  - **Run** a naive relative neighborhood graph, RNG(P, p), on all collected nearby points to determine connections to p

The binning phase creates a Poisson process with an average of one point per bin. A coarse grained plane partition heuristic is used to exclude further away points from consideration and is the method by which the process knows to stop extending the bounding cube.

Once a bounding cube has been found, other points outside of the bounding cube, but still within a, larger, local distance need to be added as they could influence

zzyzek@thumpernet.com (Z. Zzyzek)
zzyzek.github.io (Z. Zzyzek)
ORCID(s): 0009-0005-2175-1166 (Z. Zzyzek)

connections on the interior bounding cube. The list of all neighbors is given to a naive relative neighborhood graph algorithm and the process is then repeated for each point.

Given points p and q, any point, w, that lies on the other side of the plane with normal $\frac{(q-p)}{|q-p|}$ that passes through q, cannot connect to p, as q would be in any lune that p and w would create. Figure **??** gives a graphical representation of points excluded by this plane heuristic in two dimensions.

The entire region that q excludes from p is more complicated (see Figure **??**) so the plane partition heuristic is coarse but provides an easy way to exclude large portions of points.

To maintain a linear time algorithm, large collections of points that can never connect to our anchor point, p, need to be efficiently excluded. To do this, consider a current grid cell collection of integral radius $R \in \mathbb{Z}$, centered on the grid cell where anchor point p resides. If the tangent planes that passes through neighbor q, and in direction $(q - p)$ within the grid cell collection, partition each of the six sides of our cube of radius R, we can be assured that any points outside of the grid cell collection will never be able to connect to p.

A grid cell collection of a radius R is called a *fence* of radius $R \in \mathbb{Z}$. When R is understood from context, this will be called just a *fence*. If the sides of the *fence* has been partitioned, with anchor point p and neighbors q within the *fence*, the *fence* will be called *secured*.

Points outside of the *fence* will never be able to connect to anchor point p but they still might be in the lune of some points within the *fence*. For this reason, when collecting points for the naive relative neighborhood graph calculation, some points outside of the fence will need to be added, as they can sabotage potential edges within the *fence*.

This will be done by considering a larger radius of the current *fence* to add points but the increase in R will shown to be bounded by a constant factor, allowing overall linear time to be maintained.

Before discussing the algorithm in detail, preliminary structural results will be discussed.

3.2. Foundation

If w is on the other side of the plane that passes through q and with tangent in the direction of $(q - p)$, then q will always be in the lune created by p and w (see Figure **??**):

Lemma 1 (Plane Partition Heuristic). *if* $p, q, w \in \mathbb{R}^3$ *with:*

$$w \in H^+_{p,q} = \{u \ : \ (q-p) \cdot (u-q) \geq 0\}$$

then

$$q \in Lune(p, w) = \{v \ : \ |v-p| \leq |p-w| \ \& \ |v-w| \leq |p-w|\}$$

Proof. TK

The plane partition heuristic allows us to construct a convex hull from neighboring points that will bound the area that needs to be tested for relative neighborhood graph neighbors of p.

Only points within the convex hull have the potential to connect to p:

Lemma 2 (Convex Hull Heuristic). *Let* $p \in \mathbb{R}^3$, $Q = (q_0, q_1, \ldots, q_{m-1}) \subset P$ *and* $\mathbf{C}$ *be the compact convex hull created from* $H_i^- = \{u \in \mathbb{R}^3 | (q_i - p) \cdot (u - q_i) < 0\}$, *then* $(w, p) \in RNG(P, p) \rightarrow w \in \mathbf{C}$

Proof. By Lemma 1, all w outside of $\mathbf{C}$ will have some $q_i \in Q$ in the Lune(w, p), so cannot have an RNG edge to p.

Lemma 2 only guarantees that RNG connections to p fall within the hull, $\mathbf{C}$. Not every point within $\mathbf{C}$ might connect to p and points outside of $\mathbf{C}$ might still influence connections to p. For example, Figure **??** shows a potential connection between two points within the convex hull but is thwarted by a point outside the convex hull that is within the lune created by the pair.

Points outside of the convex hall might influence connections to p but we can bound how far these points can be. The maximum distance to p and the convex hull bounds what points can influence connections to p to that distance:

Lemma 3 (Convex Hull Extension Heuristic). *Let* $p, \mathbf{C}$ *be the anchor point and convex hull as detailed above. Let* $R_{max} = \max_{u \in \mathbf{C}} |u - p|$ *be the maximum distance of the convex hull boundary to p. All points that could affect* $RNG(P, p)$ *lie within a sphere of radius* R_{max}, *centered at p.*

Proof. The maximum lune radius for any neighbor to p is bounded above by $R_{\max}$. Any point, w such that $|w-p| > D$, will be further away from the maximum lune radius and so can't influence any connection within $\mathbf{C}$.

The above already suggests a linear time algorithm by starting at a grid cell where p resides and adding neighboring points from neighboring grid cells until a compact convex hull has been achieved that meets some maximum radius criteria. All points within a sphere centered at p can then be added and a naive relative neighborhood graph algorithm can be run on the reduced neighbor set. It can be shown that for a constant number of neighbors, the maximum distance of p to the convex hull is bounded, giving an expected linear time algorithm for random points in the unit cube.

While there is an expected constant number of neighbors that make up the convex hull, a natural operation to find radius bounds would be to convert to a point representation via a vertex enumeration procedure, inflating the runtime constants were a naive vertex enumeration algorithm to be used. There may be ways of speeding up the vertex enumeration by considering the structure of the problem in question but this methodology is not pursued further in this paper.

Instead, a bounding box cube is used as a proxy for the convex hull. The bounding box cube, called a *fence*, has certain key point points on each face that will be used to indicate when a cutting plane has partitioned portions of the fence from the anchor point, p. Once all portions have

been partitioned, a process that will be called *securing the fence*, there's a guarantee of an embedded convex hull within the fence and only the points within the fence need to be considered for RNG(P, p) connections.

For the anchor point p, only RNG(P, p) connections can happen within the secured fence but, as alluded to with 3, other points outside of the secured fence might influence connections within. We will see that a constant factor of fence radius need be considered to add additional points to the naive relative neighborhood graph computation.

A secured fence necessarily implies a compact convex hull embedded within the fence but the other direction is not true in general, as there can be a convex hull whose volume is completely contained within the fence but whose partition planes do not secure the fence. We will see that this has no effect on the expected linear runtime, as securing the fence for an individual anchor point is expected to use a constant number of neighbors.

TODO:

- securing fence implies compact convex hull on interior
- a secured fence need only consider a larger fence, of factor $\sqrt{3}$, to include all points that could influence relative neighborhood edges to p
- $SPoIF : RNGp(P, p)$ runs in constant expected time
- $SPoIF : RNG(P)$ runs in expected linear time

4. Algorithm

Algorithm 1 Single point naive Relative Neighborhood Graph

```
1: procedure RNGPointNaive(Q, p)
2:     for q ∈ Q/{p} do
3:         isConnected = False
4:         for u ∈ Q/{p, q} do
5:             if InLune(p, q, u) then
6:                 isConnected = False
7:         if isConnected == True then E = E ∪ (p, q)
8:     return E
```

Algorithm 2 Relative Neighborhood Graph, Secure Posts on Increasing Fence (SPoIF)

```
1: procedure RNGSPoIF(P)
2:     E = {}
3:     N = |P|,   N_s = ⌈N^(1/3)⌉,   s = 1/N_s,
4:     B[0 : N_s][0 : N_s][0 : N_s] = ∅
5:     for p ∈ P do
6:         append(B[⌊N_s p_x⌋][⌊N_s p_y⌋][⌊N_s p_z⌋], p)
7:     for p ∈ P do
8:         E = E ∪ RNGPointSPoIF(B, P, p)
9:     return E
```

Algorithm 3 Single point Relative Neighborhood Graph, Secure Posts on Increasing Fence

```
1: procedure RNGPointSPoIF(B, P, p)
2:     Initialize FencePostCluster[] entries to False
3:     Q = {},   R_f = 0,   R_max = 0
4:     c = grid cell p falls in
5:     for (R_f ≤ N_s) &(False ∈ FencePostCluster) do
6:         Q' = GridPerimeterCellPoints(B, p, R_f)/{p}
7:         for q ∈ Q' do
8:             Q = Q ∪ {q}
9:             R' = max grid cell distance from c
10:                Lune(p, q) falls in
11:            R_max = max(R_max, R')
12:        for q ∈ Q' do
13:            Define ρ(u) = sgn((q − p) · (q − u))
14:            for every cluster, FPC do
15:                if (every v ∈ FPC, ρ(v) = −ρ(p)) or
16:                    (FPC is out of bounds) then
17:                    FencePostCluster[FPC] = True
           R_f = R_f + 1
18:    for R_e = [R_f + 1 : R_max] do
19:        Q'' = GridPerimeterCellPoints(B, p, R_e)
20:        for q ∈ Q'' do
21:            Q = Q ∪ {q}
22:    return RNGPointNaive(Q, p)
```

Each cluster in FencePostCluster[] represents a small section of one of the bounding cube faces. One choice of clusters of points is a 3×3 grid of posts, evenly spaced on each of the bounding cube faces. The 3×3 posts are then grouped into neighbor 2×2 *clusters*, for a total of 4 clusters per face ($6 \cdot 4 = 24$ in total, for all six faces).

For a plane to partition a part of the bounding cube face, all posts within the cluster must be partitioned from the anchor point, which will be called *securing* the cluster. Multiple clusters can be partitioned (*secured*) by the same partioning plane, so long as individual clusters are fully partitioned. Clusters that fall out of bounds are marked as secured. Figure **??** shows an example highlighting the posts on a bounding cube face, its clusters, partitions and invalid partial partitions.

The auxiliary function, GridPerimeterCellPoints(B, p, R) returns points within the grid, B, centered at the grid cell that p resides in and that lie an integral R distance away. The list of points within the enumeration the shell of cells at distance R is returned, with cells falling completely out of bounds skipped over. A small note that the initial call of GridPerimeterCellPoints($B, p, 0$) needs to exclude anchor point, p, as the starting grid cell will include the anchor point in question.

In the worst case, Algorithm 3 will iterate until it encompases all points within the grid when the FencePostCluster[] has fallen completely out of bounds. In such a case, Algorithm 1 is run on all points and is equivalent to the degenerate condition of running a naive $O(n^2)$ relative neighborhood graph computation on all neighbor points to an anchor point.

For random point distribution in the unit cube, we expect much better runtime as the the expected number of potential neighbors collected will be bounded.

5. Conclusion

An expected linear time algorithm to find the relative neighborhood graph for 3D points placed uniformly at random in the unit cube. To the author's knowledge, no well known expected linear time algorithm for finding the relative neighborhood graph is known for the case of points distributed uniformly in the unit cube.

The algorithm presented also answers a question posed by Katajainen and Nevalainen of whether there's a simpler algorithm that wouldn't need to handle points close to the border differently. The algorithm presented avoids needing to consider side or corner regions independently as the the cut plane heuristic provides an orthogonal partition and fences near the edge of the region will be secured by falling out of bounds.

Certain pathological simple geometries can cause problems without further processing. For example, a "diamond" shaped distribution might have trouble securing the fence. This can maybe be overcome by various methods, for example doing a pre-processing step of marking grid cells as out of bounds if they fall outside of some initial convex hull calculation, or doing random rotations and processing parallel copies. These ideas are not pursued in this paper other than to mention them.

6. Bibliography

A. Appendix.0

For completeness, this section discusses the elementary, but tedious, calculations for the dart shape and volume. As a reminder, our goal is to show that increasing the grid fence by a fixed size will almost surely encapsulate a compact convex hull made from an anchor point and its neighbors. The convex hull guarantees that the relative neighborhood graph edges of the anchor point to its neighbors need only consider candidate points within the convex hull (with some caveats about extending the grid fence to include saboteurs, detailed in section **??**).

The logic chain is roughly:

- For an anchor point, p, and a grid fence side, there will be a region that a neighbor point can fall in that will partition the fence side from the point, a process we call *securing the fence*, where a secured fence necessarily implies a compact convex hull made from the points on the interior
- The volume that secures a fence side will turn out to be four sphere slices joined together, which we will call a *dart*, where a sphere slice is the intersection of two perpendicular half planes and a sphere
- A lower bound estimate on the dart volume is a constant proportion of overall grid fence volume
- A non-trivial constant dart volume makes the question of how many neighbors to be expected to secure the overal six sided fence into a coupon collector-like problem, giving us an expected finite grid fence size
- Finally, approximatations on the four slices bundled together allow us to give coarse values for grid fence sizes

It should be stressed that the discussion and estimates are meant primarily to show existence of a finite grid fence size. We focus on securing a fence side by waiting for some single neighbor point to secure it. The Shrinking Posts on Increasing Fence (SPoIF) algorithm in this paper has a smaller expected grid fence size before securitization as points can secure parts of the fence through clusters of posts, increasing the admissible region on each face side that can be secured.

The calculations done in this appendix are meant to show that we only need to consider a finite grid fence size by providing correct, if horrible, bounds. In this paper, we are satisfied with providing empherical results for grid sizes that SPoIF uses without going into more rigorous analysis.

First consider a fixed point q and a fixed but moveable point η.

We think of η as a point on a plane with normal $\eta/|\eta|$ that intersects q in the slice of $z = 0$ to $z = 1$.

As we vary η we want to know what the shape is traced out.

The claim is that this is the surface of a sphere whose center is at $q/2$ of radius $q/2$.

To see this, consider $\eta = q/2 + (|q|/2)v$, with $|v| = 1$.

Calculate dot product of the line from q to η with the line from the origin to η:

$$\begin{aligned} & \eta = q/2 + (|q|/2)v \\ & (q/2 + (|q|/2)v) \cdot (q/2 + (|q|/2)v - q) \\ = \; & (q/2 + (|q|/2)v) \cdot (-q/2 + (|q|/2)v) \\ = \; & -(1/4)|q|^2 + (|q|/2)^2|v|^2 \\ = \; & 0 \end{aligned}$$

Proving the claim.

We want to find the the minimum volume of the dart for some point inside of a fence of radius s (side length $2s$) nested inside of a larger fence of radius $s(2k+1)$ (side length $2s(2k+1)$).

First consider a sphere of radius R, centered at $(-D, -D, -D)$ intersected with two half planes, $H^+_{x=0} = \{u : u_x \geq 0\}$, $H^+_{y=0} = \{u : u_y \geq 0\}$. For suitable R and D, we want to estimate the volume.

If we take two tetrahedrons completely enclosed by the slice, the maximum diagonal of the slice is $(R - \sqrt{2}D)$, with maximum straight side length (in the x-axis line and y-axis line) to also be $(R - \sqrt{2}D)$ and slice half height to also be $(R - \sqrt{2}D)$.

The volume of the tetrahedron is then:

$$(1/6)(R - \sqrt{2}D)^2\sqrt{R^2 - 2D^2}$$

The volume of the two tetrahedrons stacked on top of each other:

$$(1/3)(R - \sqrt{2}D)^2\sqrt{R^2 - 2D^2}$$

More succinctly:

$$V_{\text{slice}} > \tfrac{1}{3}(R - \sqrt{2}D)^2\sqrt{R^2 - 2D^2}$$

We will get concrete estimates below but as a quick confirmation, we can check to make sure that this slice is a constant proportion of the fence it sits in.

We know that $R \propto (ks)$ and $D \propto (ks)$, for some $k \in \mathbb{Z}$ and grid cell size $s \in \mathbb{R}$. Call $R = \alpha(ks)$ and $D = \beta(ks)$.

The dart will be four slices bundled together sitting inside of a $\gamma^3(ks)^3$ volume. Take $\alpha, \beta \in \mathbb{R}$ to be the smallest of the four slices:

$$\begin{aligned}
&\frac{(4/3)(\alpha(ks)-\sqrt{2}\beta(ks))^2\sqrt{\alpha^2(ks)^2-2\beta^2(ks)^2}}{\gamma^3(ks)^3} \\
= \quad &\frac{(4/3)(ks)^3(\alpha-\sqrt{2}\beta)^2\sqrt{\alpha^2-2\beta^2}}{\gamma^3(ks)^3} \\
= \quad &\frac{(4/3)(\alpha-\sqrt{2}\beta)^2\sqrt{\alpha^2-2\beta^2}}{\gamma^3}
\end{aligned}$$

as desired.

When choosing random points inside of the larger fence cube, we can think of the dart as the probability a single point secures one side of the fence. It is equivalent to talk about securing one side of the fence face with securing four posts on each of corners of the fence face. As a reminder, the Shrinking Posts on Increasing Fence (SPoIF) uses nine posts per face, equally distributed along the face square. Since there are more opportunities to secure clusters of posts on a fence face, in addition to still retaining the features of the dart region, SPoIF will be able to secure the fence cube more quickly. We only consider the four post fence here to help simplify the calculation even though it inflates the expected number of points we need to see to secure the grid fence.

Take the minimum dart value volume of the six directions and call it V_{dart}. The probability that a point lands in one of the darts is bounded below by $V_{\text{dart}}/V_{\text{fence}}$. Of the points that land in one of the darts, the time we need to take to secure each face of the fence is now a coupon collector-like problem with six coupons.

We can now plug in some real numbers to get coarse bounds on dart volumes, probabilities of landing in the critical region and expected grid sizes.

As a reminder, we are considering a center grid cell, of side length s where the anchor point, p, resides, all of which is inside a larger grid fence of side length $2s(k+1)$, for some fixed $k \geq 0, k \in \mathbb{Z}$ as yet to be determined.

The smallest size a slice can be is if the anchor point is on the corner of the interior grid cell and to choose the grid fence corner nearest to it. For example, $p = (s/2, s/2, s/2)$, $q = (s(k + 1/2), s(k + 1/2), s(k + 1/2))$.

In such a case, the center point of the sphere is:

$$(p + q)/2 = (s(k + 1)/2, s(k + 1)/2, s(k + 1)/2)$$

with radius:

$$\begin{aligned}
R &= |(p + q)/2 - p| \\
&= |(sk/2, sk/2, sk/2)| \\
&= (\sqrt{3}/2)sk
\end{aligned}$$

Here:

$$\begin{aligned}
V_{\text{fence}} &= (s(2k + 1))^3 \\
D &= s(k + 1)/2 - s/2 \\
&= sk/2
\end{aligned}$$

This yields:

$$\begin{aligned}
\frac{V_{\text{dart}}}{V_{\text{fence}}} > \quad &\frac{\frac{4}{3}(\frac{\sqrt{3}}{2}sk-\frac{\sqrt{2}}{2}sk)^2(\frac{3}{4}s^2k^2-\frac{1}{2}s^2k^2)^{\frac{1}{2}}}{s^3(2k+1)^3} \\
= \quad &\frac{s^3k^3(\sqrt{3}-\sqrt{2})^2}{3\sqrt{2}s^3(2k+1)^3} \\
= \quad &\frac{(\sqrt{3}-\sqrt{2})^2}{3\sqrt{2}(2+\frac{1}{k})^3} \\
> \quad &\frac{(\sqrt{3}-\sqrt{2})^2}{81\sqrt{2}} \qquad (k \geq 1) \\
> \quad &1/1198
\end{aligned}$$

For points that land within one of the six dart region, we would expect this to be be a six coupon collector problem with, at worst, equal probability. The coupon collector problem grows roughly as $n\log(n)$ which, in our case of six sides of the fence to secure, gives us an expected 11 or more points before we secure the fence.

Each point has roughly a $6/1198$ chance of landing in one of the six darts, so we would expect roughly $11/(6/1198) = 2196.3\ldots$ number of points within the grid fence to appear before securing the fence.

Each grid cell has roughly one point, so we would expect the number grid cells on a side to be

$$\begin{aligned}
&(11 \cdot 1198/6)^{\frac{1}{3}} \\
&= 12.9\ldots \\
&\sim 13
\end{aligned}$$

That is, we would expect $k \sim 6$, iterating roughly six to seven steps of increasing the grid fence size until securing the fence in this way.

B. Appendix.1

... Appendix sections are coded under `\appendix`.

References

...